# Lab 6: Transforming Commands in Splunk

## Lab Overview

In this lab, you will master Splunk's transforming commands to derive meaningful statistics and insights from your data. You'll work with real-world scenarios involving web application logs, database audit trails, and system logs to understand how transforming commands can help answer business questions.

## Learning Objectives

By the end of this lab, you will be able to:

- Use the `top` command to identify most frequent values
- Apply the `rare` command to find uncommon events
- Leverage the `stats` command with various statistical functions
- Create reports with count, distinct count, sum, and average calculations
- Rename fields and format output for better readability
- Sort and limit results for focused analysis
- Combine multiple transforming commands in a search pipeline

---

## 🚀 Welcome to Content 2!

In this section you'll learn advanced Splunk analysis and reporting techniques. You'll build on the fundamental search skills from Content 1 to create statistical reports, dashboards, and automated monitoring.

---

## Prerequisites

- Completed Labs 1-4 (Content 1)
- Basic understanding of Splunk search syntax
- Familiarity with field extraction and filtering

## Lab Environment Setup

### Step 1: Verify Sample Data

Before starting, ensure you have the required sample data loaded:

```
index=main earliest=-30d latest=now
| stats count by sourcetype
```

You should see these sourcetypes:

- `access_combined_wcookie` - Web application logs
- `db_audit` - Database audit logs
- `linux_secure` - Linux security logs

If data is missing, return to Lab 1 to load the sample data.

### Step 2: Set Search Time Range

For all exercises in this lab:

1. Navigate to **Search & Reporting**
2. Set time range to **Last 30 days**
3. Ensure you're searching in the `main` index

> **Data window**: The course dataset covers roughly **26 days** ending at the moment `labs/data/generate_course_data.py` was last run — deliberately sized to sit inside Splunk's **Last 30 days** preset. Set the time picker to **Last 30 days** for every exercise unless a step says otherwise.
>
> Regenerate the data within **4 days** of delivering the class; past that the preset starts clipping the oldest events and the answer-key counts drift.

## Key Concepts

### Transforming Commands Overview

Transforming commands take search results and transform them into statistical data tables. Key commands include:

- **top/rare**: Find most/least common values
- **stats**: Calculate statistics (count, sum, avg, etc.)
- **chart**: Create data series for visualization
- **timechart**: Create time-based data series
- **transaction**: Group related events

### Field Reference

Throughout this lab, you'll work with these key fields:

| Sourcetype | Key Fields |
|---|---|
| access_combined_wcookie | action, bytes, categoryId, clientip, JSESSIONID, productId, referer, status, file |
| db_audit | Command, Duration, Type |
| linux_secure | raw syslog text — extract `user` / `src_ip` with `rex` (see Lab 7) |

---

## Exercise 1: Analyzing Best-Selling Products with `top`

**Scenario**: The sales team needs a report of the five best-selling products.

### Task 1.1: Find Successful Purchases

First, identify events where purchases were completed:

```
index=main sourcetype=access_combined_wcookie
  status=200
  file=success.do
```

**Expected Results**: Events showing successful purchase transactions

### Task 1.2: Get Top Products

Add the `top` command to find best-selling products:

```
index=main sourcetype=access_combined_wcookie
  status=200
  file=success.do
| top productId
```

**Validation**: You should see 10 products with counts and percentages

### Task 1.3: Limit Results

Modify to show only top 5 products:

```
index=main sourcetype=access_combined_wcookie
  status=200
  file=success.do
| top productId limit=5
```

### Task 1.4: Remove Percentages

Clean up the display:

```
index=main sourcetype=access_combined_wcookie
  status=200
  file=success.do
| top productId limit=5 showperc=false
```

✓ **Checkpoint**: Record the #1 best-selling productId: _____

---

## Exercise 2: Security Analysis with `rare`

**Scenario**: The security team wants to identify rarely accessed files that might indicate backdoors or unauthorized access.

### Task 2.1: Find All Successful File Access

```
index=main sourcetype=access_combined_wcookie
  status=200
```

### Task 2.2: Identify Rare Files

```
index=main sourcetype=access_combined_wcookie
  status=200
| rare file
```

**Analysis Question**: Do you see any suspicious file names?

### Task 2.3: Add Time Context

Break down rare files by month:

```
index=main sourcetype=access_combined_wcookie
  status=200
| rare file by date_month
```

✓ **Checkpoint**: List any files that appear only 1-2 times: _____

---

## Exercise 3: Shopping Cart Analytics with `stats count`

**Scenario**: Marketing wants to understand the conversion funnel from cart additions to purchases.

### Task 3.1: Find Cart and Purchase Events

```
index=main sourcetype=access_combined_wcookie
  (file=cart.do OR file=success.do)
  status=200
```

### Task 3.2: Count by Action Type

```
index=main sourcetype=access_combined_wcookie
  (file=cart.do OR file=success.do)
  status=200
| stats count by file
```

### Task 3.3: Improve Readability

```
index=main sourcetype=access_combined_wcookie
  (file=cart.do OR file=success.do)
  status=200
| stats count as Transactions by file
| rename file as Function
```

✓ **Checkpoint**:

- Cart additions: _____
- Successful purchases: _____
- Conversion rate: _____%

---

## Exercise 4: User Session Analysis with `stats dc()`

**Scenario**: Management needs to understand user login patterns.

### Task 4.1: Count Unique Sessions

```
index=main sourcetype=access_combined_wcookie
| stats dc(JSESSIONID)
```

### Task 4.2: Add Label

```
index=main sourcetype=access_combined_wcookie
| stats dc(JSESSIONID) as Logins
```

### Task 4.3: Break Down by IP

```
index=main sourcetype=access_combined_wcookie
| stats dc(JSESSIONID) as Logins by clientip
```

### Task 4.4: Find Top Users

```
index=main sourcetype=access_combined_wcookie
| stats dc(JSESSIONID) as Logins by clientip
| sort –Logins
```

✓ **Checkpoint**: IP with most logins: _____

---

## Exercise 5: Bandwidth Analysis with `stats sum()`

**Scenario**: IT needs to understand bandwidth consumption by different application functions.

### Task 5.1: Calculate Total Bandwidth

```
index=main sourcetype=access_combined_wcookie
  status=200
| stats sum(bytes) as TotalBytes
```

### Task 5.2: Break Down by File

```
index=main sourcetype=access_combined_wcookie
  status=200
| stats sum(bytes) as TotalBytes by file
```

### Task 5.3: Sort Results

```
index=main sourcetype=access_combined_wcookie
  status=200
| stats sum(bytes) as TotalBytes by file
| sort file
```

✓ **Checkpoint**: File using least bandwidth: _____

---

## Exercise 6: Database Performance Analysis with `stats avg()`

**Scenario**: EMERGENCY! The website is slow. A developer suspects a database query issue.

### Task 6.1: Calculate Average Query Time

```
index=main sourcetype=db_audit
| stats avg(Duration)
```

### Task 6.2: Break Down by Command

```
index=main sourcetype=db_audit
| rex field=Command "^(?<sql_verb>\w+)"
| stats avg(Duration) as "time to complete" by sql_verb
```

> **Note**: Each `Command` value contains its own literal IDs, so grouping by `Command` yields tens of thousands of single-event rows. Extract the SQL verb first to get a meaningful breakdown.

### Task 6.3: Find Slowest Queries

```
index=main sourcetype=db_audit
| rex field=Command "^(?<sql_verb>\w+)"
| stats avg(Duration) as "time to complete" by sql_verb
| sort -"time to complete"
```

✓ **Checkpoint**: What pattern do you notice in the slowest commands?

---

## Exercise 7: Browser Analytics with `stats list()` and `values()`

**Scenario**: The web team needs a browser usage report.

### Task 7.1: List All User Agents

```
index=main sourcetype=access_combined_wcookie
| stats list(useragent)
```

**Observation**: Notice duplicate entries

### Task 7.2: Get Unique Values

```
index=main sourcetype=access_combined_wcookie
| stats values(useragent) as "Agents used"
```

### Task 7.3: Add Usage Counts

```
index=main sourcetype=access_combined_wcookie
| stats count as "Times used" by useragent
| sort -"Times used"
```

### Task 7.4: Format as Table

```
index=main sourcetype=access_combined_wcookie
| stats count as "Times used" by useragent
| sort -"Times used"
| rename useragent as "Browser Agent"
| table "Browser Agent", "Times used"
```

---

## Challenge Exercises

### Challenge 1: Multi-Metric Dashboard

Create a single search that shows:

- Top 5 products sold
- Total revenue (assuming bytes represents price)
- Average transaction size
- Number of unique customers

```
index=main sourcetype=access_combined_wcookie
  status=200 file=success.do
| stats count as Sales,
        sum(bytes) as Revenue,
        avg(bytes) as "Avg Transaction",
        dc(clientip) as "Unique Customers" by productId
| sort -Sales
| head 5
```

### Challenge 2: Time-Based Analysis

Analyze purchase patterns over time:

```
index=main sourcetype=access_combined_wcookie
  status=200 file=success.do
| bin _time span=1d
| stats count as Purchases,
        dc(clientip) as Customers by _time
| eval "Purchases per Customer"=round(Purchases/Customers,2)
```

### Challenge 3: Correlation Analysis

Find correlation between file access and client location:

```
index=main sourcetype=access_combined_wcookie
| stats count by file, clientip
| stats list(clientip) as IPs,
        dc(clientip) as "Unique IPs",
        sum(count) as "Total Accesses" by file
| sort -"Unique IPs"
```

---

## Troubleshooting Guide

### Common Issues and Solutions

| Issue | Solution |
|---|---|
| No results returned | Check time range is set to "Last 30 days" |
| Fields not found | Verify field names are case-sensitive |
| Stats showing 0 | Ensure status=200 filter is included |
| Percentages showing when not wanted | Add `showperc=false` to top command |
| Sort not working | Use `-` prefix for descending sort |

### Performance Tips

1. **Use specific indexes**: Always specify `index=main`
2. **Filter early**: Apply filters before transforming commands
3. **Limit time range**: Use smallest time range possible in production
4. **Use field aliases**: Create reusable field aliases for complex extractions

---

## Cleanup

No cleanup required for this lab as we're only running searches, not creating any persistent objects.

---

## Key Takeaways

✅ **You've learned to:**

- Use `top` and `rare` for frequency analysis
- Apply various `stats` functions for calculations
- Format and present data professionally
- Combine commands for complex analysis
- Optimize searches for performance

## Next Steps

- Proceed to Lab 7: Creating Reports and Dashboards
- Practice combining these commands with visualization
- Explore additional stats functions like stdev, median, and mode

---

## Quick Reference

### Essential Transforming Commands

| Command | Purpose | Example |
|---|---|---|
| top | Find most common values | `top field limit=10` |
| rare | Find least common values | `rare field by category` |
| stats | Calculate statistics | `stats count, sum(field), avg(field)` |
| chart | Create data tables | `chart count over field by category` |
| timechart | Create time-based charts | `timechart span=1h count` |

## Common Stats Functions

| Function | Description | Syntax |
|---|---|---|
| count | Count events | `stats count` |
| dc() | Distinct count | `stats dc(field)` |
| sum() | Sum values | `stats sum(field)` |
| avg() | Average | `stats avg(field)` |
| min() | Minimum value | `stats min(field)` |
| max() | Maximum value | `stats max(field)` |
| list() | List all values | `stats list(field)` |
| values() | List unique values | `stats values(field)` |

## Answer Key

🔑 **Instructor answer key**

### Exercise 1

- Best-selling product: WSC-MG-G10 (SIM Cubicle Tee), 204 units

### Exercise 2

- Rarest files with status=200: oldlink (1,710), success.do (3,054), cart.do (14,092)
- Note: `logo.ico` never appears here — those requests always return 404.
- Answer to "any suspicious file names?": no. This dataset contains only ordinary application paths; the exercise teaches the `rare` workflow you would use to spot an anomaly in production data.

### Exercise 3

- cart.do events (status=200): 14,092
- success.do events (status=200): 3,054

### Exercise 4

- IP with most sessions: 92.46.53.223 (3,309 distinct JSESSIONIDs)
- Runners-up: 212.58.253.71 (2,830), 91.214.92.22 (2,390)

### Exercise 5

- File using least bandwidth: oldlink (~3.6 MB)
- Heaviest: product.screen (~111.0 MB)

### Exercise 6

- Average duration by command type: UPDATE 54.8, INSERT 54.6, SELECT 27.4, DELETE 17.5
- Pattern: write operations take roughly twice as long as reads